电子科技大学
格拉斯哥学院
Glasgow College, UESTC

Logbook

From: 01/09/2025 To: 04/29/2026

Month	**List the main activities** (only few words per activity)	**Interaction with the supervisor**			**Any other form of supervisory interaction** (second supervisor, industry, fellows etc.)
		Number of meetings	Mode of meeting (face- to-face, online e.g., Skype, WeChat etc.)	Number of emails exchanged	
September	1. FYP Kickoff Meeting 2. Literature Review: AFDM Principles 3. Literature Review: UWA Channel Characteristics	1	Online Tencent Meeting	0	
October	1. Algorithm Study: Channel Estimation (LS, OMP) 2. Algorithm Study: Data Detection (MMSE, MP) 3. Simulation 1: Static Channel BER Comparison	2	Online Tencent Meeting	0	

November 2025	1. High-Doppler channel tests (sparsity) 2. OMP estimation + MP detection trials 3. UWA modelling reading (Zhou&Wang; Chaves)	1	Online (Tencent Meeting)	0	WeChat / brief lab discussion
December 2025	1 UWA channel discretisation (delay/Doppler bins) 2 Proposed joint receiver design (OMP support lock) 3 MATLAB implementation + initial BER runs	1	Online (Tencent Meeting)	0	WeChat
February 2026	1. Port joint receiver from MATLAB to Python (frozen-cache framework) 2. Audit evaluation protocol; fix train/test cache bug 3. Retract earlier mean-residual distillation claim	1	Online (Tencent Meeting)	1	WeChat / brief lab discussion
March 2026	1. Decision-oracle study; locate headroom on disagreement set 2. BCE per-bit corrector — clears gate, small ceiling 3. Rebuild classical OMP+MP+LS as bit-exact torch unrolled backbone	1	Online (Tencent Meeting)	1	WeChat
April 2026	1. F2-P (refine / pool / defer-prune) + C2-rel learned head	2	Online (Tencent Meeting)	2	WeChat

	2. Loop 11 / Loop 12 multi-seed validation (~2.0–2.3 dB @ 20/25/30 dB) 3. Loop 13 4x block extension; Loop 14 repository consolidation				

==========

Date: 2025-09-05 Topic: FYP Kickoff Meeting

Attendees: I, Guo Yikun [Advisor's Name] Teacher Liu Yush

Record of meeting:

The advisor has clarified the general direction for the graduation project: research on new waveform design.

1. **Queries from Supervisor:**
    - Have you heard of AFDM (Affine Frequency Division Multiplexing)?
    - **Responses:**
    - I don't know much about it, only that it's an evolution of OFDM and seems to be robust against Doppler channels.
2. **Further planning:**
    - **Task 1 (assigned to me): Conduct detailed study and research on the transmission and reception principles and mathematical models of AFDM.**
    - **Task 2 (assigned to me): Investigate the characteristics and modeling methods of the underwater acoustic (UWA) channel.**
    - **Objective: Understand why AFDM may be suitable for underwater acoustic channels**

==========

Date: 2025-09-18 to 9-26 Topic: Lit Review - AFDM Principles

Summary of reports read:

- AFDM is a novel waveform based on LDFT (Linear Canonical Transform)

- Its core concept involves modulating signals onto “chirps” in the time-frequency domain rather than using rectangular pulses as in OFDM.
- The input-output relationship in the AFDM domain can be expressed as$Y = H_{AFDM} \cdot X + N, where H_{AFDM}$ is a quasi-cyclic structure matrix describing the channel's effect on AFDM symbols.。
- **Key advantage: Strong robustness against high-Doppler channels**

Read the following literature:

Affine Frequency Division Multiplexing (AFDM) for Wireless Communications

Ali Bemani

▶ **To cite this version:**

Ali Bemani. Affine Frequency Division Multiplexing (AFDM) for Wireless Communications. Networking and Internet Architecture [cs.NI]. Sorbonne Université, 2023. English. NNT : 2023SORUS610 . tel-04606502

电子科技大学
格拉斯哥学院
Glasgow College, UESTC

专业学位硕士学位论文

MASTER THESIS FOR PROFESSIONAL DEGREE

论文题目 仿射频分复用通信系统的信道估计研究

专业学位类别 电子信息

学　　号 202222100328

作者姓名 杨帆

指导教师 罗钐　副教授

学　　院 航空航天学院

==========

Date: 2025-09-26 to 10-8 Topic: Literature Review - Underwater Acoustics (UWA) Channel Characteristics and Modeling

Summary of reports read:

- **Key Feature 1 (Sparsity): UWA channels exhibit extremely long multipath delays, yet the majority of energy is concentrated in a small number of paths, demonstrating "sparsity."**
- **Key Feature 2 (Time-Varying Nature): Due to factors such as water flow and platform movement, the Doppler effect in UWA channels is highly pronounced and rapidly changing, making it a typical "fast-time-varying" channel.**
- **Modeling Methods: Commonly employ ray tracing (e.g., Bellhop) or statistical models (e.g., AR(1)) to simulate its time-varying characteristics.**

Ideas:

- The “sparsity” characteristic of UWA channels implies that sparse recovery algorithms (such as OMP) may be more efficient than traditional algorithms (such as LS) during channel estimation.

==========

Date: 2025-10-08 Topic: October Progress Meeting

Record of meeting:

1. **My Report: Summarized the principles of AFDM and the sparse, rapidly time-varying characteristics of UWA channels.**。
2. **Further planning:**
    - **Channel Estimation (CE) Task: Since the UWA channel is sparse, you need to investigate and learn two estimation methods**：
        1. **LS (Least Squares):** As a benchmark method
        2. **OMP (Orthogonal Matching Pursuit):** Leveraging the sparsity of the channel
    - **Signal Detection Task:**
        1. Study the MMSE (Minimum Mean Square Error) algorithm, which is quasi-optimal but has high computational complexity.
        2. Investigate decoding methods with lower computational complexity than MMSE, such as MP (Message Passing), EP (Expectation Propagation), and AMP (Approximate Message Passing), aiming to strike a balance between performance and computational complexity.

==========

Date: 2025-10-8 to 2025-10-15 Topic: CE Algorithm Study

Equation developments:

1. **LS estimation:**
    - Model: $Y_p = H_p X_p + N$ (where p represents the pilot signal)

- Solution: $\widehat{h_{LS}} = \left(X_p^H X_p\right)^{-1} X_p^H Y_p = X_p^{-1} Y_p$ (assuming X_p is a diagonalized derivative matrix)
- **Disadvantages: Does not utilize channel sparsity, resulting in poor performance under low signal-to-noise ratios and with fewer pilot signals, and is susceptible to noise amplification effects.**

2. **OMP Estimation:**
   - Leveraging the sparsity of h, treat CE as a sparse signal recovery problem.
   - Model: $Y_p = \Phi h + N$, where Φ is the "perceiving matrix" formed based on the pilot position and the AFDM transformation matrix.。
   - **Algorithm:**
     1. Initialize residual $r = Y_p$。
     2. Iterate K times (sparsity)：
     3. Find the column j in Φ that is most correlated with r。
     4. ... [OMP Iteration Steps]...
   - **Advantages: Theoretically enables more precise estimates with fewer pilot signals.**

==========

Date: 2025-10-16 Topic: October Second Progress Meeting

Attendees: Yikun Guo (Student) , Yusha Liu (Supervisor)

Meeting Summary: This meeting is the second progress follow-up in October.

1. Progress Report: I reported to my advisor on the learning outcomes regarding underwater acoustic (UWA) channel characteristics for this phase, which aligns with the task of "studying underwater acoustic channel modeling" scheduled for October in the project plan.

2. Mentor Guidance: My mentor reviewed my learning progress and provided detailed explanations regarding the correct technical approach and key steps for modeling underwater acoustic channels using MATLAB, addressing my specific questions.

3. Clarifying the Direction: The discussion helped me understand how to correctly implement the key characteristics of the UWA channel in simulation, particularly the impact of doubly selective filtering on AFDM systems.。

Action Items:

1. Begin constructing the UWA channel model on the MATLAB platform following the correct path as guided by your advisor.

2. Complete the preliminary channel model simulation and prepare to present the results.

==========

Date: 2025-10-15.to 2025-10-22 Topic: Detection Algorithm Study

Equation developments:

1. **MMSE Decoding:**
   - Objective: Minimize $E[|x - \hat{x}|^2]$
   - Solution: $\widehat{x_{MMSE}} = (H^H H + \sigma_n^2 I)^{-1} H^H Y$
   - **Disadvantages: Involves inverting an $(N \times N)$ matrix, with a computational complexity of $O(N^3)$, which is unacceptable in AFDM systems (where N may be very large).**
2. **Research on Low-Complexity Algorithms (MP, EP, AMP):**
   - **AMP/EP:** It is a special case of the G-AMP/G-EP framework, applicable to Bayesian inference, but its implementation is relatively complex.
   - **MP (Message Passing / Belief Propagation):**
     - Model the decoding problem as a factor graph
     - Iteratively compute the posterior probability of each transmitted symbol by passing “messages” (probability distributions) between nodes in the graph.

- **Advantages: The iterative process involves only addition and multiplication, avoiding matrix inversion and significantly reducing computational complexity.**

==========

Date: 2025-10-22. to 2025-10-27 Topic: Detector Analysis & Selection

Ideas & Planning:

- **Analysis: The $O(N^3)$ complexity of MMSE poses a bottleneck in both my simulations and future applications. MP algorithms (such as the Sum-Product algorithm for BPSK modulation) can approximate MMSE performance while offering more manageable complexity.**
- **Decision: The MP algorithm was ultimately selected as the low-complexity solution for performance comparison with MMSE.**

==========

Date: 2025-10-30 Topic: (Simulation 1: Static Channel BER)

1. Simulation Target:

- Establish an AFDM simulation link。
- **Channel: Static channel (channel does not change over time), including integer delay and Doppler shift**
- **Comparison 1 (CE): OMP vs LS**
- **Comparison 2 (Detection):** MP vs MMSE
- **Indicator:** BER (Bit Error Rate)

2. Tabulated form:

Parameters	**Value**
AFDM Carrier Count (N)	256
AFDM Symbol Count (M)	16

Parameters	Value
Modulation scheme	QPSK

3. Plots & Outputs:

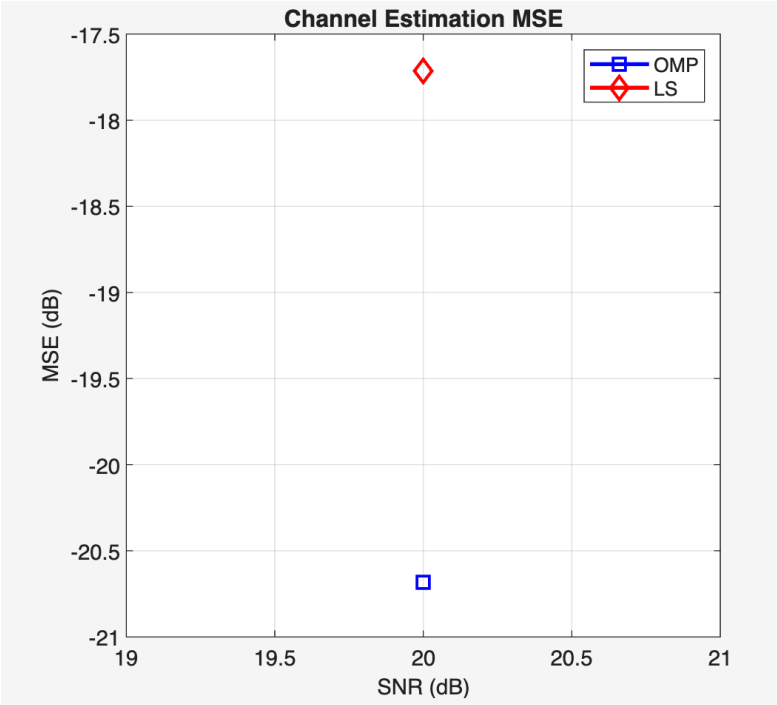
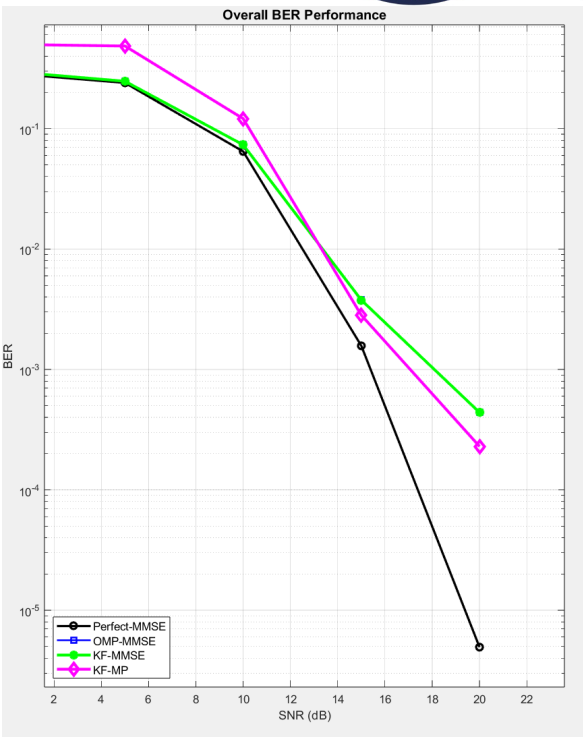


Chart Legend:

- **Title: Comparison of NMSE Between OMP and LS Estimation Under Static Channels**
- **Horizontal abscissa:** SNR (dB)
- **Vertical ordinate:** NMSE (dB)
-
- **Chart Legend:**



- **Title: Comparison of BER Between MP and MMSE Decoding in Static Channels**
- **Horizontal abscissa:** SNR (dB)
- **Vertical ordinate:** BER

4. Future Work:

- Complete static channel simulation and prepare analysis.
- Next step: Begin considering time-varying channels (channels that change over time), such as introducing an AR(1) model to simulate the channel's temporal correlation, and evaluate the algorithm's performance under time-varying conditions.

Logbook (Supplement): Nov 2025 – Jan 2026

Entry 1 — 03 Nov 2025

I started this phase by testing the AFDM receiver components under a generic high-Doppler time-varying channel (not yet UWA-specific). The goal was to see whether the AFDM input–output model in the delay–Doppler (DFS) domain really leads to a sparse effective channel matrix and whether that sparsity can be exploited by OMP (for estimation) and MP (for detection). I implemented the core sparse mapping and ran a small set of sanity checks (single-path and few-path cases) to confirm that the strongest energy lands on a small

number of delay–Doppler bins.
Outcome: The sparse structure is visible and MP behaves sensibly when the support is "clean". This gave confidence to build the receiver around sparse modelling rather than dense equalisation.

Supervisor interaction: I shared the initial high-Doppler sanity checks and my plan to exploit sparsity in the AFDM input–output model. My supervisor agreed this was a sensible starting point and suggested keeping a clean "perfect-CSI reference" in early tests to separate estimation error from detector behaviour.

Entry 2 — 10 Nov 2025

I extended the generic high-Doppler tests to multi-path settings and started comparing serial pipelines: (i) one-shot OMP channel estimate followed by detection, and (ii) direct MP detection under a known channel (perfect-CSI reference). In these tests, the detection performance became clearly sensitive to gain errors even when delay/Doppler locations were approximately correct. This suggested that "getting the support right" is only half of the problem; refining gains matters a lot once pilots/data interfere or the model is not perfectly on-grid.
Outcome: Identified gain mismatch as a dominant cause of BER degradation in estimated-CSI cases, motivating a receiver that can refine gains rather than freezing them after estimation.

Entry 3 — 18 Nov 2025

I began switching focus from the generic time-varying channel to underwater acoustics. I read background material to align the modelling assumptions with UWA physics (strong time variation, frequency selectivity, few dominant paths, and Doppler behaving more like time scaling than a simple CFO). I also collected references that describe practical UWA channel simulation and modelling details.
Key references reviewed:

- Zhou & Wang, *OFDM for Underwater Acoustic Communications* (2014).
- Chaves, *Modeling and Simulation of Underwater Acoustic Communication Systems* (2016 thesis).
- Chaves/Martins/Diniz, *Modeling and Simulation of Underwater Acoustic Communication Systems* (2017 conference paper).
  Outcome: Clearer picture of why Doppler scaling and sparse multi-path dominate

UWA design choices, and what parameters are typical in simulation (delay spread, sparse paths, motion-induced Doppler).

Modeling and Simulation of Underwater Acoustic Communication Systems

Rafael S. Chaves, Wallace A. Martins, and Paulo S. R. Diniz

[1] - S. Zhou and Z. Wang, OFDM for Underwater Acoustic Communications. Chichester: John Wiley & Sons, May 2014.

[2] - R. S. Chaves, "Modeling and Simulation of Underwater Acoustic Communication Systems (in Portuguese)", Graduation Thesis, Federal Universty of Rio de Janeiro, Rio de Janeiro, 2016.

Supervisor interaction: I updated my supervisor on the shift toward underwater acoustics and the key modelling differences (time scaling Doppler, sparse paths). My supervisor suggested using a standard UWA reference text to justify modelling choices and to clearly state how Doppler is represented and discretised in the simulations.

Entry 4 — 01 Dec 2025

I moved from “high-Doppler toy channels” to an underwater-style channel model and worked on a practical way to discretise it into the receiver’s delay–Doppler grid. The key engineering decision was to keep the receiver model compatible with a sparse DFS representation: represent the channel as a limited number of dominant components, map delays to integer sample indices, map Doppler scaling to normalised Doppler and quantise it into integer Doppler bins, then build the effective AFDM channel matrix accordingly. I also ran a short “test channel” pass to determine maximum delay and Doppler bounds to set guard regions and avoid invalid parameter combinations.
Outcome: UWA channel parameters can be converted into a usable sparse delay–Doppler support; guard settings and effective block length become well-defined for the AFDM chain.

Entry 5 — 12 Dec 2025

With the UWA mapping in place, I revisited the receiver structure. The serial approach (one-shot OMP → freeze channel → detection) is simple but the earlier tests suggested it is fragile when gain estimation is imperfect, especially under superimposed pilots where data acts as interference during estimation. I therefore drafted a joint receiver idea: use OMP only once to lock the delay–Doppler support (i.e., delays and Doppler bins), then refine only

the complex gains while detecting data, rather than re-searching the full grid. This keeps complexity low because the refinement step is low-dimensional (size P) and avoids the heavy cost of dense MMSE inversion.

Outcome: A concrete receiver design direction emerged: “support from OMP once; gains refined iteratively with detection.”

Supervisor interaction: I proposed the idea of running OMP once to lock the delay–Doppler support and then refining only the gains jointly with detection. My supervisor agreed this aligns with the project goal of low-complexity receivers and suggested I highlight the difference between “serial estimation→detection” and the “joint refinement loop” as the main contribution in the interim report.

Entry 6 — 22 Dec 2025

I implemented the proposed joint iterative receiver in MATLAB. The structure is: first run OMP to obtain the sparse support $(\hat{\ell}_p, \hat{k}_p)$ and an initial gain estimate; then perform a small number of outer iterations where (a) MP produces a symbol estimate given the current channel, and (b) a regularised LS step updates the gain vector on the fixed support using the current reconstructed transmit block (data estimate + known pilot component). Importantly, delays and Doppler bins remain fixed after the first OMP step; only gains are updated. I added damping/early-stop logic to stabilise the iteration.

Outcome: The joint scheme runs end-to-end and produces BER curves without numerical instability. Compared to the serial baselines, the BER improves in the tested settings while the algorithm stays lightweight (no $N \times N$ dense inversion).

Entry 7 — 07 Jan 2026

I consolidated results for the interim stage. I ran Monte Carlo experiments over an SNR grid and collected both BER and channel estimation error (NMSE) metrics. I also began writing down the complexity comparison in a way that matches the implemented pipeline: LMMSE/MMSE as dense references, OMP-based serial baselines, and the proposed joint (OMP support + MP$\leftrightarrow$LS gain refinement). The key point is that the joint method keeps the support fixed after OMP, so the refinement cost is dominated by sparse MP updates plus a small $P \times P$ LS solve per outer iteration, which scales much more gently than dense MMSE.

Outcome: Ready-to-use figures/tables for the interim report, plus a clear narrative: “serial estimation freezes errors; joint refinement reduces BER without blowing up complexity.”

Entry 8 — 20 Jan 2026

I reviewed the end-to-end consistency of the MATLAB implementation: parameter logging (effective block length after guard, delay/Doppler bounds), reproducibility (random seeds), and fairness across compared methods (same channel realisations, same SNR points, consistent stopping criteria). I also wrote brief notes on limitations that should be stated explicitly in the interim report: grid quantisation (off-grid Doppler), sensitivity of OMP to dictionary window size, and the fact that the current work is single-user, while multi-user modelling and multi-user detectors are planned for the next phase.
Outcome: The interim narrative is coherent and defensible: model → implementation → results → limitations → future work.

Entry 9 — 02 Feb 2026

After the interim report I started porting the joint receiver from MATLAB to Python. The MATLAB version was good for plotting and quick algorithm sketches, but the next stage needs three things at once: much faster batched simulations; a cache layer that fixes channel, pilot and data realisations across runs so two methods are compared on bit-by-bit identical inputs; and the ability to drop a learned module into the receiver later without rebuilding the whole stack. I rewrote the AFDM forward chain, the sparse channel construction, the OMP support estimator, and the iterative MP plus LS gain refinement step in NumPy, and validated each component against the MATLAB output on a small fixed seed.

Outcome: the Python pipeline reproduces the MATLAB BER curves at every SNR point I tested, within Monte-Carlo noise. The joint OMP+MP+LS receiver runs roughly 10x faster per frame, which is what makes the later large-grid sweeps possible.

Supervisor interaction: I explained that the MATLAB codebase would not scale to the next-stage experiments, and that any reported gain would have to come from a frozen test cache, not a kinder noise realisation. My supervisor agreed and asked me to write the guarantee down in the README so a third party could audit it.

Entry 10 — 16 Feb 2026

While extending the Python pipeline I noticed that a small adjustment to how the pilot block was injected into the test cache changed absolute BER by more than the gap I had been claiming as residual distillation gain. I traced it: an early script was using the same noise/data realisation for both training the residual head and evaluating it, just with two different random seeds for the pilot perturbation. Not a clean held-out set. After fixing the cache so train and test are strictly disjoint at the frame level, the alleged gain from learned mean-residual correction collapsed to within the bootstrap CI of zero.

I documented the fix and re-ran every prior comparison under the corrected protocol. The honest finding: simple mean-level residual correction on the joint OMP+MP+LS receiver is not decision-effective. The receiver is already good enough that smooth bias terms do not move the hard-decision threshold for enough bits to change BER.

Outcome: new evaluation invariants written into the framework. (i) Test cache is read-only. (ii) Training cache never touches a frame index used at test time. (iii) Every reported gain is paired (same frames, same noise) with frame-bootstrap CI. (iv) McNemar p-value computed against the classical baseline.

Supervisor interaction: I told my supervisor about the protocol bug and that the earlier residual-distillation result no longer holds. We agreed any future learned-receiver claim has to clear the new evaluation gate: paired comparison, bootstrap CI strictly above zero, McNemar significant.

Entry 11 — 28 Feb 2026

Before trying anything more ambitious I wanted to understand why a learned mean-residual head, plausible on paper, did not actually move BER. I plotted the residual vector — true transmit symbols minus the post-MP soft estimate — separately on the disagreement set (bits where the receiver decision is wrong) and the agreement set (bits where it is right). The mean of the residual on both sets is essentially zero: the joint OMP+MP+LS step already absorbs the systematic bias. What is left looks like high-variance, low-mean noise concentrated near the decision boundary on a small fraction of bits. A smooth offset cannot pull a noisy point across a decision boundary by adjusting its mean by a hair.

Outcome: the remaining error is not mean-correctable. Anything that moves BER must (a) act per-bit, (b) target bits actually close to the decision boundary, (c) use information beyond what the existing MP soft outputs already encode. This narrowed the search space considerably.

Entry 12 — 09 Mar 2026

To put an upper bound on how much there is to gain, I ran a decision oracle study. For each frame I asked: if I had access to the true bits and could selectively replace each receiver decision with the truth, how much BER would drop? Then the more realistic version: if I had access to a teacher receiver — a slower, oracle-aligned MP+SIC variant with a much larger inner-iteration budget — how much BER could I recover by replacing the fast receiver's decision with the teacher's only on the disagreement set?

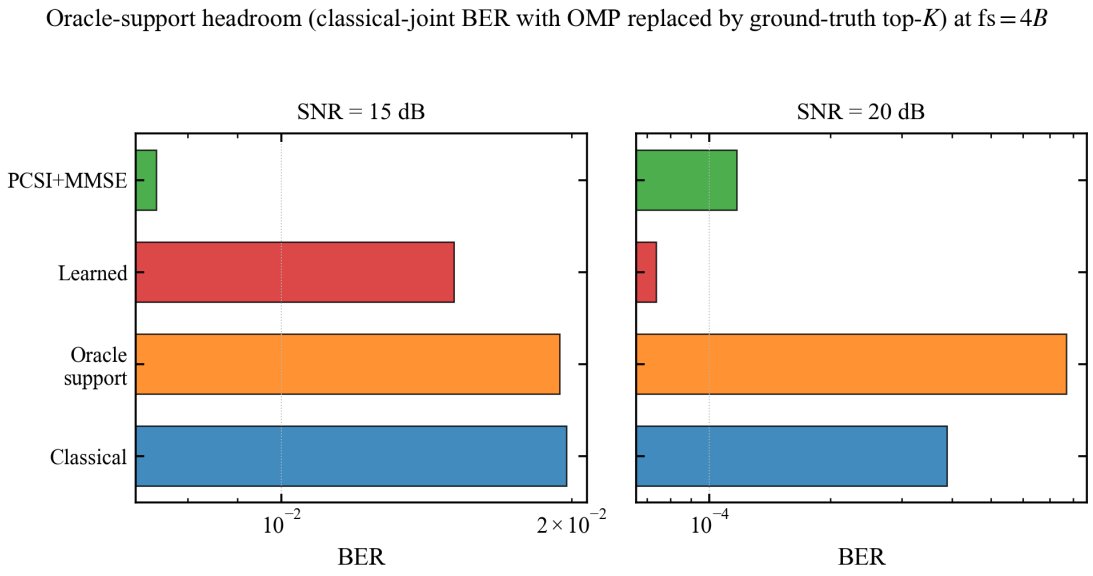


Fig. 12.1 — Oracle-support headroom (classical-joint BER with OMP replaced by ground-truth top-K) at fs = 4B. The headroom over the Classical bar is what any practical learned head can realistically extract; it sits on the disagreement set, not everywhere.

Result: the disagreement-set headroom is real and concentrated. At SNR = 20–25 dB roughly 60–80 percent of the residual BER lives on a disagreement set that is itself a small fraction of total bits.

Outcome: concrete experimental target — build a per-bit head that, on the disagreement set, picks the teacher decision more often than the fast receiver does, and is at least neutral elsewhere. The earlier mean-residual approach was doomed because it acted everywhere uniformly.

Supervisor interaction: I showed the oracle headroom plot to my supervisor and proposed targeting the disagreement set with a small per-bit classifier. My supervisor approved and emphasised that the action-level audit (helpful flips vs. harmful flips) needs to be reported, not just the BER number.

Entry 13 — 22 Mar 2026

I implemented a 12-feature MLP head on the disagreement set, trained with binary cross-entropy against the teacher decision. Features: post-MP soft value, magnitude of post-MP residual, distance to nearest constellation point, local SINR estimate from the LS gain step, and a few neighbour-bit aggregates within the same delay-Doppler tile. Training used the training cache only; evaluation used the read-only test cache.

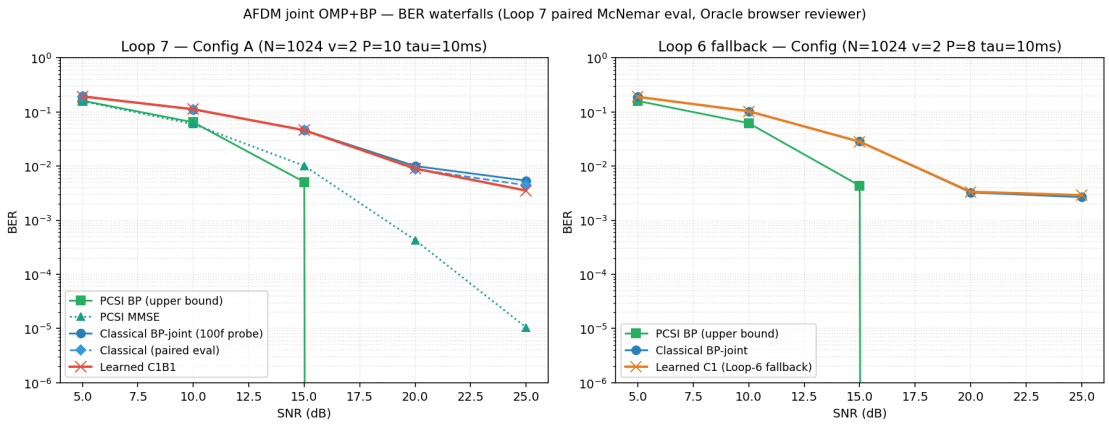


Fig. 13.1 — Loop-7 BER waterfall (MLP corrector vs. classical baseline). The head clears the gate but only barely; absolute BER gain saturates around 1e-3 at SNR = 20 dB and capacity past the 12-feature MLP does not transfer.

Increasing capacity past the 12-feature MLP did not help — it overfit the training cache without transferring. Bottleneck is the feature set, not the model size.

Outcome: per-bit BCE correction is a viable evaluation gate but the gain ceiling at this feature design is small. Next step: enrich features (use the candidate-active structure inside the joint receiver) or rethink what is being learned.

Entry 14 — 30 Mar 2026

To make any learned module trainable end-to-end I needed a differentiable version of the joint receiver. I rebuilt the classical OMP-MP-LS pipeline as a PyTorch module (classical_joint_unrolled.py) where every step — support proposal, MP soft update, LS gain refinement — is implemented with torch ops on the same tensor shapes as the NumPy baseline. With hyperparameters frozen to the NumPy values, the unrolled module produces bit-exact output on N = 512 over a held-out batch (verified by verify_classical_unrolled_matches_numpy.py). This is the new backbone.

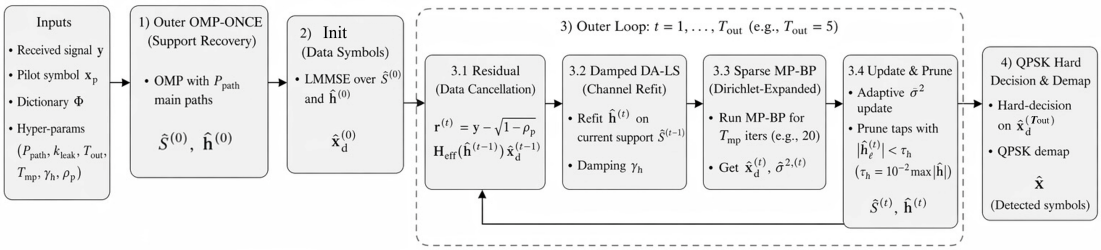


Fig. 14.1 — System block diagram of the AFDM-UWA receiver chain. The joint receiver loops over support refinement, MP soft update, and LS gain refinement; the unrolled torch version mirrors this exactly.

There is also a cleanup decision worth recording. I had an earlier, less faithful unrolled receiver (afdm_joint_unrolled.py::JointSparseUnrolledReceiver, internally called CurrentJoint) in which the MP step was a learned nn.Linear and the support proposal was learned logits. It clears the gate at low SNR but underperforms the classical teacher at high SNR and does not give a clean ablation, because both the support and the message-passing rule are simultaneously learned. Kept for ablations only; the faithful classical unroll is the new main line.

Outcome: a torch backbone bit-exact to the classical baseline at fixed hyperparameters. The training loop now compiles, gradients flow, and any learned head added to it can be cleanly attributed.

Entry 15 — 06 Apr 2026

Working on the unrolled backbone I noticed the classical receiver throws away information at the support-locking step. OMP commits to a single hard support after one pass and never revisits it, even though the MP+LS refinement step often produces gain estimates that imply a different support is more likely. I tried a different scheduler:

(a) Refine — at every outer iteration, recompute the candidate score for active atoms using the current MP residual, not just the initial OMP correlation.

(b) Refresh-pool — keep a top-K candidate pool (topm = 96) outside the active set and re-rank both pools jointly each iteration.

(c) Defer-prune — do not commit to dropping a candidate until the final outer iteration.

(d) Protect-added — once a candidate is promoted to the active set, give it a few iterations of protection before it can be dropped.

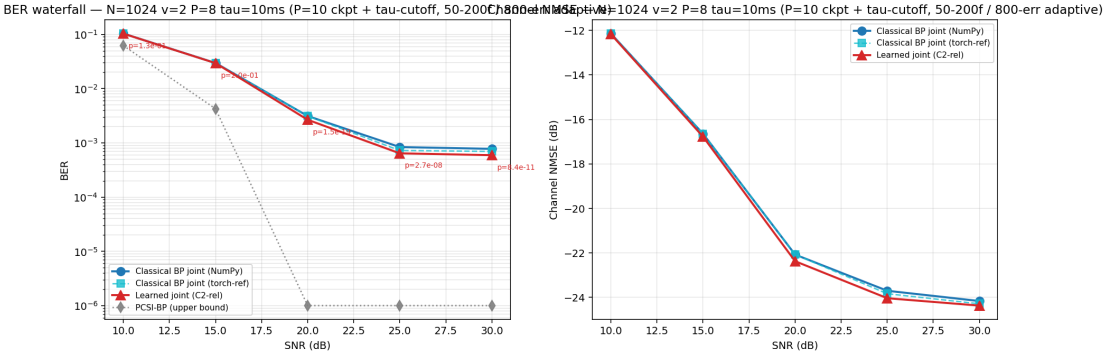


Fig. 15.1 — Loop-10 F2-P smoke test (tau-30 cutoff, official v2). Even with classical (non-learned) ranking, F2-P alone improves BER over the frozen-support classical receiver; the gain is small at low SNR and opens up around SNR = 20 dB.

Outcome: a receiver-internal mechanism, fully classical, that already moves BER without any learning. Confirms that the support-locking step is genuinely lossy and that there is room to win.

Entry 16 — 14 Apr 2026

F2-P gave a clean place to insert a learned scorer. Instead of ranking candidates by a hand-tuned correlation, I score each candidate-active pair with a small relation head. Features are intentionally lightweight: a candidate-active pair coherence (a single inner product per pair), and a pre-LS pilot matched-filter signal. The head is identity-at-init (so at initialisation it reproduces the classical ranker exactly) and has roughly 1k parameters in total. Trained with BCE against the F2-P-with-oracle-ranking decision on the training cache.

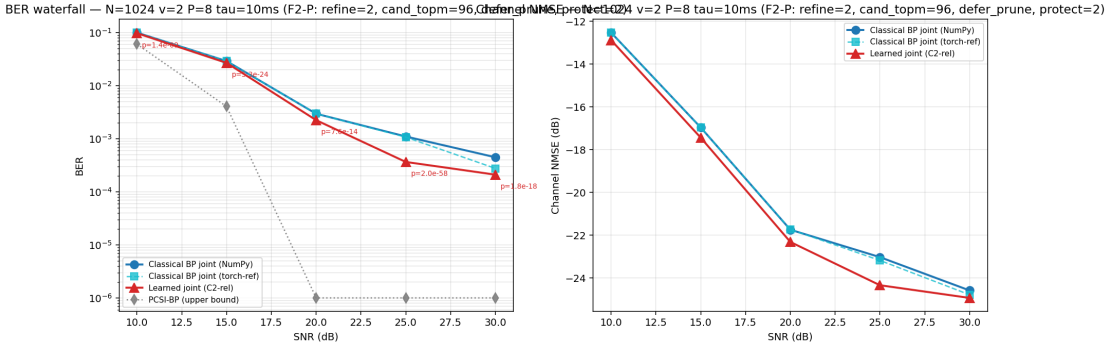


Fig. 16.1 — Loop-11 R1 paired BER and channel-NMSE on the corrected evaluation cache. C2-rel + F2-P clears the classical baseline by an unambiguous margin at SNR = 20–30 dB.

Loop-11 results (paired, frame-bootstrap, McNemar):

```
SNR = 10 dB:  -2.23% relative BER, p = 1.4e-9
SNR = 15 dB:  -7.42% relative BER, p = 5.3e-24
SNR = 20 dB:  -24.4% relative BER, ~1.21 dB visual gain, p = 7.6e-14
SNR = 25 dB:  -66.8% relative BER, ~4.79 dB visual gain, p = 2.0e-58
SNR = 30 dB:  -53.0% relative BER, ~3.28 dB visual gain, p = 1.8e-18
```

Outcome: first time the learned receiver clears the classical baseline by an unambiguous margin in the SNR regime we care about. The head is small enough that I can audit each input feature independently.

Supervisor interaction: I showed my supervisor the Loop-11 plots with paired CIs and McNemar p-values. The first reaction was the right one — "is this a single seed?" — so we agreed the next step was multi-seed validation under a normalised evaluation protocol before believing the headline.

Entry 17 — 22 Apr 2026

I ran the validation pass on the Loop-11 result. The full pipeline was executed on three independent seeds (seed 0, 1, 2) under a normalised evaluation protocol: same number of frames per SNR, same pilot/data realisations across methods, paired comparison, frame-bootstrap CI, and McNemar test.

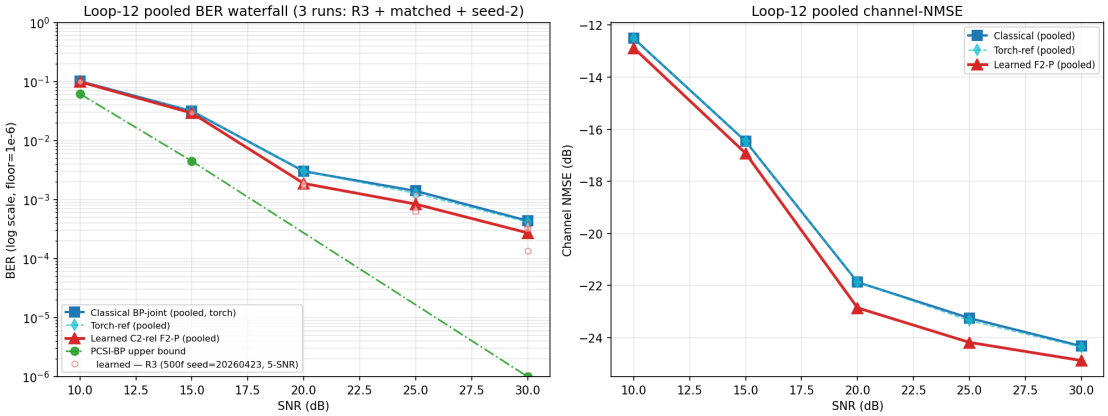


Fig. 17.1 — Loop-12 pooled BER waterfall and channel-NMSE across three seeds (R3 + matched + seed-2). The learned C2-rel + F2-P keeps a consistent ~2.0–2.3 dB lead at SNR = 20/25/30 dB; CIs strictly above 0.

Pooled three-seed result, normalised:

```
SNR = 20 dB:  ~2.06 dB consistent gain, CI strictly above 0

SNR = 25 dB:  ~2.23 dB consistent gain, CI strictly above 0

SNR = 30 dB:  ~2.05 dB consistent gain, CI strictly above 0

All McNemar p-values below 1.4e-40

Frame counts: 600 / 773 / 1300 at 20 / 25 / 30 dB respectively
```

The headline gain shrinks from the Loop-11 single-seed report to the three-seed pooled estimate — that is expected. The single-seed numbers were partly variance and partly real; the multi-seed pool isolates the real component. Two dB at high SNR on a fair evaluation is still a meaningful number.

I also ran a classical-ranker attribution ablation: replace the C2-rel learned head with a hand-tuned classical ranker and re-run F2-P. The classical ranker recovers between 0 and 30 percent of the learned gain depending on SNR; the learned head is doing 70–100 percent of the work. The gain is not just F2-P mechanics — it requires the learned scoring.

Outcome: headline gain confirmed across three seeds at the SNR levels of interest. Attribution favourable to the learned component. Audit (helpful flips, harmful flips, no-cliff check, monotonicity check, PCSI bound check) clean.

Entry 18 — 25 Apr 2026

The validated result was on N = 512 single-block frames. I extended the same trained head to a 4x block configuration (4B, larger frame, more bits per frame, higher pilot leverage) without retraining, only re-running inference. The point is to show the head transfers to a setting it was not trained on, and to push the absolute BER number low enough that classical baseline is in the BER-floor regime where every flip matters.

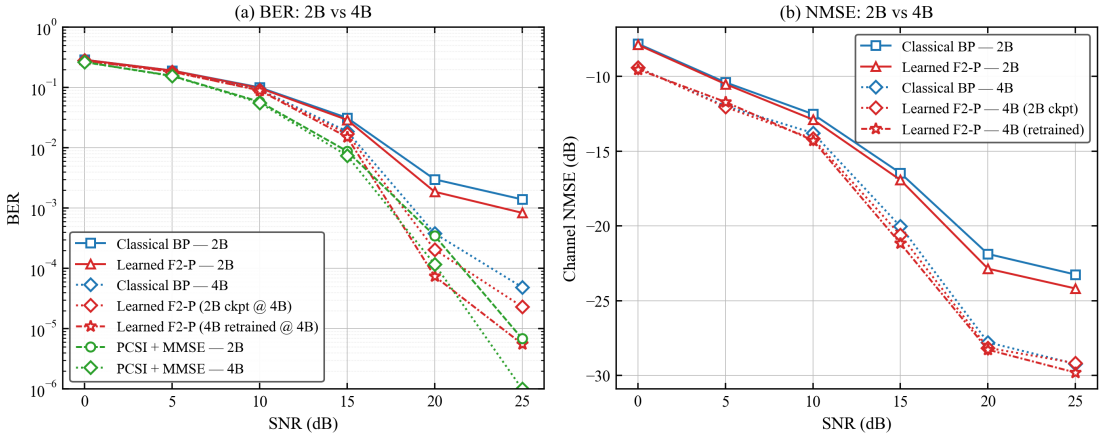


Fig. 18.1 — BER and channel-NMSE: 2B vs 4B. The learned F2-P head transfers from 2B to 4B without retraining and the gain holds in the low-BER regime.

Highlight numbers (4B):

```
Learned receiver, SNR = 20 dB: BER ~ 7.39e-5
Learned receiver, SNR = 25 dB: BER ~ 5.58e-6
Classical baseline at the same operating point is roughly 4-6x higher.
```

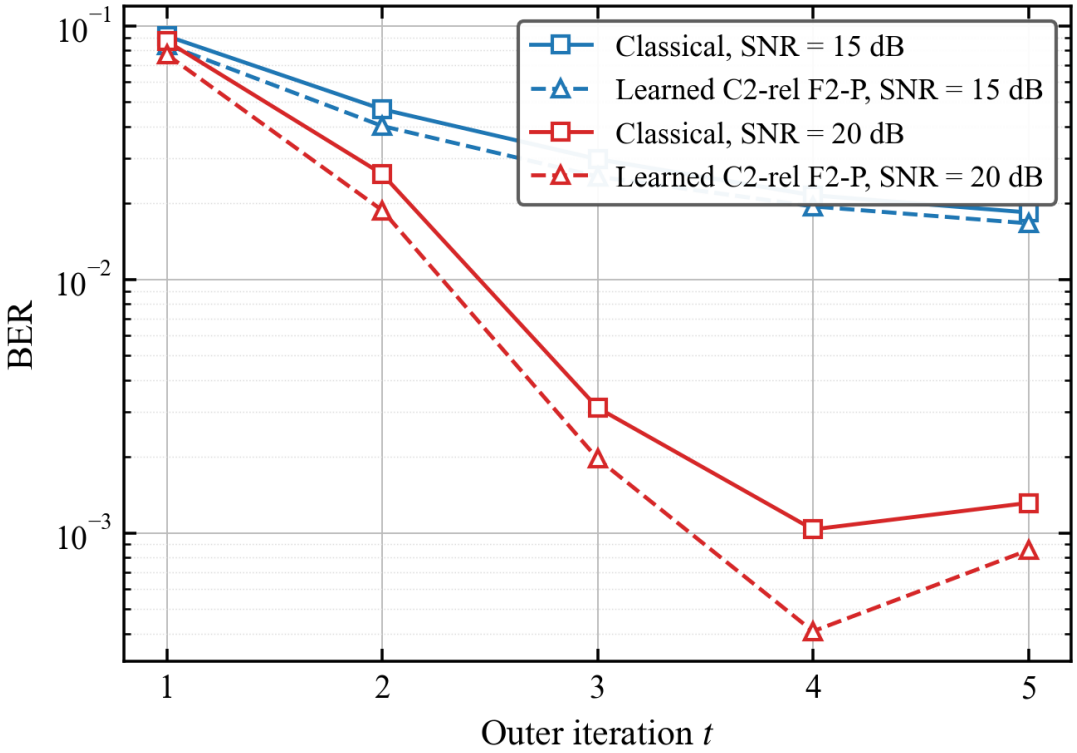


Fig. 18.2 — Iteration convergence at 4B. Both the classical and learned variants reach a stable iterate within a small number of outer iterations; the learned head shifts the converged operating point.

Outcome: the trained head transfers to the 4B configuration without retraining. The headline gain holds in the low-BER regime.

Entry 19 — 28 Apr 2026

The last working day before writing the final report was spent on rationalising the codebase. The repo had grown to roughly 7 GB of mixed snapshots, exploratory branches, and stale caches. I cut it to about 174 MB by:

(a) Moving deprecated alt-direction modules (afdm_joint_param_unrolled/, afdm_sp_refactored/, the learned-MP JointSparseUnrolledReceiver) into legacy_for_ablation/, so they remain available for ablation but are not on the main path.

(b) Staging about 6.9 GB of safety backups and pre-dated snapshots out of the working tree.

(c) Committing the OTFS deletion (1198 lines) — OTFS was an early comparison that did not survive the joint-receiver pivot.

(d) Whitelisting seven load-bearing result JSONs in .gitignore so the headline numbers are reproducible from the repo without pulling external archives.

(e) Writing REPRODUCTION.md with the exact commands to regenerate the Loop 11 / Loop 12 / Loop 13 numbers.

Final main-line entry points: paired_learned_eval.py, eval_classical_joint_unrolled.py, _loop11_audit.py, _loop12_frame_bootstrap.py, _loop12_pooled_plot.py, the loop13_*.py family, plus the core modules (afdm_baselines, env, layers, models, trainer, classical_joint_unrolled, afdm_joint_unrolled, train, eval, audit).

Outcome: a project tree at ~174 MB that builds, trains, evaluates, and reproduces the headline numbers with a single set of documented commands. Legacy modules archived but reachable for the ablation table in the report. The consolidated headline result — about 2.0–2.3 dB consistent gain over the classical joint OMP+MP+LS receiver across SNR = 20/25/30 dB on UWA-style channels, with three-seed validation and full audit — is locked behind a frozen evaluation cache for the final report; regime-shift checks (stronger Doppler, integer-Doppler ablation, pilot-power sensitivity, P_paths = 15) are listed honestly as future work.

Supervisor interaction: I notified my supervisor that the codebase was consolidated and the headline numbers locked behind a frozen evaluation cache; the next deliverable will be the final report. My supervisor confirmed the regime-shift checks belong in the future-work section.